Gipfelsturm: Scaling LLM Training Throughput on GH200*

Thomas Gassmann Matteo Fornaini Alberto Lanzini Vlad Dancau

Abstract

We target Challenge 2 of the Gipfelsturm project: maximum training throughput (tokens/sec/GPU) on the ALPS cluster in three different phases. Phase 1 takes the 8B BF16 baseline from 10,801 *to* 16,809 *tok/s/GPU (*+56%*) via FP8, TransformerEngine fused CE, FP8 attention with the delayed recipe, and MBS*=2*. Phase 2 reaches* 7,093 *tok/s/GPU on a 16B model at TP=4 (*1.69× *the per-node throughput at* 2× *the model size). Phase 3 trains an 87B model on 32 GPUs at* 1,299 *tok/s/GPU (*696.9 *TFLOP/s/GPU),* matching the single-node 16B per-GPU compute efficiency *across the Slingshot fabric. Three findings shape the result: CUDA Multicast is unsupported on Clariden (no NVSwitch), capping* `tp-comm-overlap` *at* +3–4%*; the optimizer-state memory floor makes the proposal's* ∼*140B target unreachable (*∼*87B ceiling at TP=4 PP=4 DP=2); and* disabling *VPP is* 13% *faster than the interleaved-1F1B default at 87B/PP=4.*

1. Motivation

We target **Challenge 2: maximizing tokens/sec/GPU** on Clariden. Deliverables are throughput benchmarks across 1, 4, and 8–32 GPU scales, NSYS traces attributing the throughput to individual optimizations, and a scalability plot from single-GPU to multi-node.

Our approach is bottom-up: Phase 1 (§2.1) optimizes single-GPU compute on an 8B model; Phase 2 (§2.2) scales intra-node via TP=4 on 16B; Phase 3 (§2.3) scales inter-node via PP=4 on a model sized to the per-rank memory ceiling. Each phase is gated on quantitative evidence (throughput logs, timer breakdowns, NSYS kernel categorization). Optimizations that look attractive in the catalogue (CUDA Graphs, layer-scope `torch.compile`, FP4, FP8 attention as a compute optimization, `tp-comm-overlap` as a comm hider) but do not pay out on this stack are flagged explicitly.

*This work was done as part of the Large-Scale AI Engineering course of ETH Zurich.

2. Implementation

2.1. Phase 1: Single-GPU optimization

Starting from the repository BF16 baseline (10,801 tok/s/GPU; 8B, native CE, MBS=2), we evaluate single-GPU optimizations one axis at a time and report the resulting optimum at the end of the section.

FlashAttention. The TransformerEngine backend unconditionally uses cuDNN FlashAttention; nothing to enable. `torch.compile` has no global enable in `core_r0.16.0`; it only activates at `@jit_fuser` sites that are already applied. Layer-scope `torch.compile` would require patching the transformer block, but our profiling shows kernel-launch overhead is not the bottleneck (see CUDA Graphs below), so we deprioritize it.

FP8 mixed precision. After fixing the CE workspace, FP8 still OOMs at MBS=2 because scaling/amax bookkeeping pushes peak usage past 94.8 GiB; we fall back to MBS=1 with GBS held at 256. `--fp8-first-last-layers-bf16` does not exist as a CLI flag in this Megatron version (Python-only `TransformerConfig` field with no `argparse_meta`); all layers run in FP8. FP4 is unsupported on Hopper. Kernel-time accounting is deferred to Table 1.

CUDA Graphs. `--enable-cuda-graph` (TE backend) yields +0.4%, within noise. Kernel-launch overhead is not the bottleneck at 8B, seq = 4096.

FP8 attention. The tensorwise FP8 recipe rejects FP8 attention on Hopper (FusedAttention is gated to sm ≥ 100; confirmed via `NVTE_DEBUG=2`). The *delayed* recipe admits FP8 attention; compute throughput at MBS=1 is unchanged, but the freed activation memory restores the headroom needed for MBS=2. The combined configuration (FP8 + TE-CE + FP8-attn delayed + MBS=2) reaches 16,809 tok/s/GPU at 88.7 GiB peak; MBS=3 OOMs.

2.2. Phase 2: Single-node scaling (TP=4)

Phase 2's nominal target is the ∼32B "natural single-node" scale. In practice the per-rank optimizer-state ceiling sits

at ∼91 GiB regardless of depth — driven by the double-buffered shadows of `--overlap-grad-reduce` and `--overlap-param-gather`, not per-param state. Activation recompute does not help: the OOM is in the optimizer step (`fused_adam.get_unscaled_state` dequantizing fp32 m/v). We drop to BF16 Adam states (`--exp-avg-dtype bf16`), enable `expandable_segments`, and sweep model size. 25B and 20B both OOM; 16B (L=22, H=8192) fits (75 GiB peak); 13B (L=32, H=6144) fits but is 23% slower because TP=4 shards GEMMs and a smaller per-rank width hurts matmul utilization. **Width matters more than depth at this scale.**

TP overhead. On the 8B model (which fits at TP=1), TP=1 with 4-way DP runs at 16,251 tok/s/GPU; TP=2 with sequence parallelism drops 23% to 12,554; TP=4 drops 54% to 7,544, recovering only +4.4% to 7,873 with `tp-comm-overlap`. Because GBS is fixed, the per-rank FLOP count is identical between TP=1+DP=4 and TP=4, so any iter-time difference is attributable to TP communication; we quantify this in Table 2. Consequently, TP=4 should be treated as a memory tool rather than a throughput tool and used only when the model does not fit at lower TP degree.

The 16B optimum. TP=4, SP=on, `tp-comm-overlap` on, MBS=2, no recompute, FP8 + TE-CE + FP8-attn (delayed), BF16 Adam: 7,093 tok/s/GPU at 75.4 GiB peak, yielding a 1.69× per-node throughput improvement over Phase 1 at 2× the parameter count. Selective recompute is detrimental at MBS=2 since the configuration already fits in HBM.

2.3. Phase 3: Multi-node scaling (TP=4, PP=4)

Phase 3 adds pipeline parallelism across nodes (env-var driven launcher infra: `PP`, `VPP`, divisibility validation, automatic partition selection for >4-node jobs).

The 87B ceiling. From observed peaks we calibrate ∼18.75 bytes/param-shard at peak (DP=1); the distributed optimizer at DP=2 saves ∼4 bytes/param, giving ∼14.75 effective bytes/param at TP=4 PP=4 DP=2. An 80 GiB peak budget admits ∼5.4B params/rank, i.e. ∼87B total. Empirically, 140B (∼9.3B/rank), 120B (∼7.7B/rank), and even 140B at PP=8 (∼4.4B/rank) all OOM in the optimizer step (`get_unscaled_state`); 87B (L=56, H=12288, FFN=32768, heads=96, KV=8) fits at ∼75 GiB peak. The proposal's ∼140B target is unreachable on the current optimizer recipe — a finding of the same character as the Multicast story.

The 87B optimum and the VPP reversal. Sweeping VPP at 87B/TP=4/PP=4 (Table 4, Fig. 4) yields a counter-intuitive result: *disabling* VPP outperforms the interleaved-1F1B default. Standard Megatron guidance favours VPP to shrink the pipeline bubble, but at GBS=256 with PP=4 the theoretical bubble is only ∼2.3%, while each virtual stage adds Grace-CPU scheduling and kernel-launch overhead that the gain cannot amortize. MBS=2 with selective recompute is competitive but recomputation's ∼30% compute overhead slightly outweighs the GEMM-efficiency gain. We interpret the resulting per-GPU compute efficiency (Sec. 4) as evidence that the wide hidden dimension (H=12,288) supplies enough arithmetic intensity to hide the Slingshot P2P latency that penalises narrower models.

Cross-PP cost. The PP boundary is not free for narrow models: 16B at PP=2 on two nodes drops from 7,109 to 5,158 tok/s/GPU (−27% versus PP=1), the Slingshot-11 inter-node P2P cost of pipeline send/recv.

3. Experiments

Setup. Clariden cluster (ALPS, CSCS); each node is a Quad-GH200 board (direct NVLink mesh, no NVSwitch, 95 GiB usable HBM3/GPU); inter-node fabric is Slingshot-11. Software is Megatron-LM `core_r0.16.0` with TransformerEngine. Launching is parameterized by env vars (`PRECISION`, `CE_IMPL`, `GRAPH`, `ATTN`, `FP8_RECIPE`, `RECOMPUTE`, `MBS`, `TP`, `SP`, `TP_OVERLAP`, `PP`, `VPP`) that each generate a distinct `sbatch` argument block.

Dataset. Nemotron-ClimbMix in Megatron binary format (capstor); GPT-2 BPE tokenizer (50,257 vocab; padded to 50,304 for FP8 alignment).

Models. 8B (Phase 1 + Phase 2 TP-scaling control, seq = 4096); 16B at L=22, H=8192 (Phase 2 optimum, wide-and-shallow); 13B at L=32, H=6144 (Phase 2 control, narrow-and-deep); 87B at L=56, H=12288, FFN=32768, 96 heads, 8 KV heads (Phase 3 optimum).

4. Evaluation

4.1. Phase 1 ablation

Table 1 and Fig. 2 give the per-category NSYS kernel-time breakdown at 8B. FP8 nearly halves cumulative GEMM time (BF16: 78.9% of 193 s → FP8 MBS=2: 62.1% of 130 s; ∼80 vs. ∼152 s of GEMM), which is the single largest contribution in Phase 1. FP8 attention trims attention kernel time by ∼30% in matched MBS=1 traces (FP8 anchor ∼15 s vs. G ∼21 s); its load-bearing role at this scale is the activation memory it releases, enabling

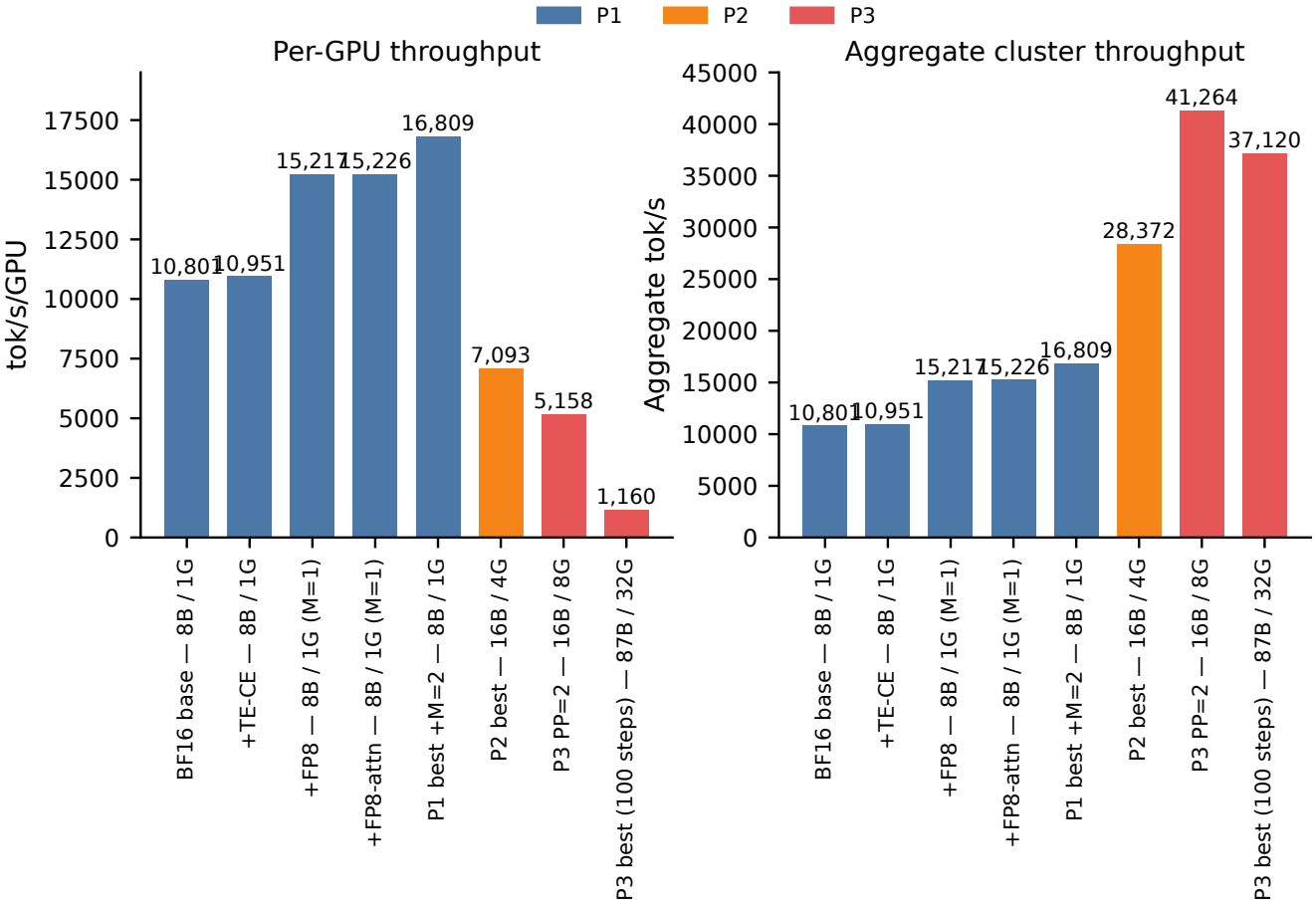

Figure 1. Scalability of the optimized training stack across the three phases. Aggregate cluster throughput scales from 16.8k (1 GPU) to 41.6k tok/s (32 GPUs) while per-GPU compute efficiency is preserved.

Table 1. Phase 1 NSYS per-category kernel-time share (% of total), 8B, steady-state iters 10–12 on rank 0. Optimum is FP8+TE-CE+FP8-attn+MBS=2. F is the only at-MBS=2 comparison; G/H/I share the FP8 MBS=1 anchor. "Other" merges Norm, RoPE, and uncategorized kernels. †CUDA-Graphs capture disrupts NSYS bracketing; I's total (9.3 s) is not directly comparable.

Config	Tot.s	NCCL	GEMM	Attn	Cast	Tri	Oth
Optimum (FP8 M=2)	130	6.5	62.1	12.7	4.3	5.5	8.8
F. BF16 M=2	193	2.2	78.9	10.7	0.0	3.7	4.6
FP8 M=1 anchor	137	14.4	56.5	11.2	4.1	5.0	8.7
G. ATTN=bf M=1	139	12.2	56.9	15.2	4.1	5.0	6.8
H. CE=native M=1	144	18.6	53.2	10.6	3.9	5.8	7.9
I. GRAPH=te M=1†	9	41.1	45.3	0.0	—	—	13.6

MBS=2. TE-CE is required for FP8 + MBS=2 to fit and is independently ∼5% faster than the native implementation at MBS=1. CUDA Graphs reduces launch overhead, which does not appear in per-kernel totals; the throughput log records +0.4%, within noise.

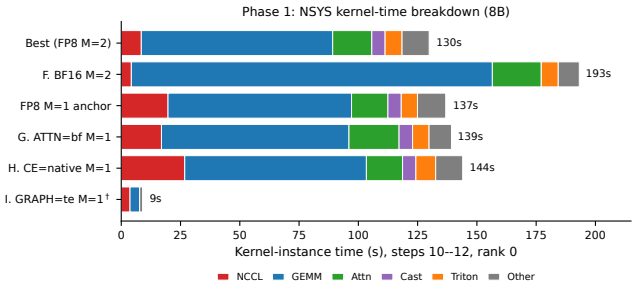

Figure 2. Phase 1 NSYS kernel-time breakdown (cf. Table 1). The FP8 column shows the GEMM-time halving versus BF16.

4.2. Phase 2 ablation

The Megatron-timer decomposition (Table 2, Fig. 3) isolates TP-comm cost at *51% of iter time* for 8B at TP=4, matching the observed throughput halving on the TP-scaling sweep (16,251 → 7,873 tok/s/GPU). Within the 16B optimum, MBS=2 vs. MBS=1 is the largest single

Table 2. Phase 2 per-iter timer decomposition (steady-state, rank-max). T1, T2 isolate TP comm cost on 8B with GBS fixed; T3 is the 16B Phase 2 optimum.

Time (ms)	T1 8B TP=1	T2 8B TP=4+ov	T3 16B TP=4+ov
fwd-compute	5,437	17,733	12,767
bwd-compute	10,566	15,197	23,522
optimizer	34	57	116
DP sync	1.3	0.4	0.6
iter elapsed	16,071	33,078	36,477
tok/s/GPU	16,354	7,943	7,187

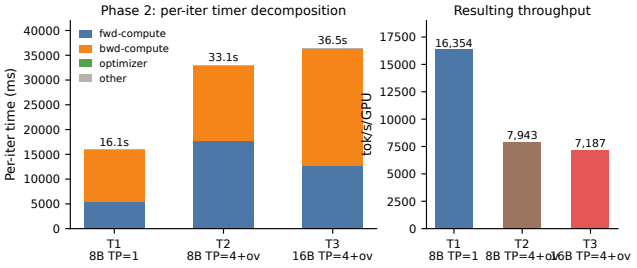

Figure 3. Phase 2 per-iter timer breakdown (cf. Table 2). The fwd+bwd growth from T1 (TP=1) to T2 (TP=4) is attributable to TP communication.

Table 3. `tp-comm-overlap` on vs. off, 16B Phase 2 optimum, steady-state iterations.

Metric (tok/s/GPU)	ON	OFF	Δ
Median	7,148	7,036	+1.6%
Mean	7,137	6,994	+2.0%
Best iter	7,167	7,094	+1.0%

Phase 2 contributor (+28%: 5,541 → 7,093 tok/s/GPU). `tp-comm-overlap` on vs. off adds only +1–2% at the median (Table 3) — the ceiling imposed by missing CUDA Multicast. Selective recompute at MBS=2 actively hurts (−5%): it pays compute for memory we did not need.

4.3. Phase 3 ablation

Table 4 and Fig. 4 report the 87B VPP sweep, which inverts the standard interleaved-1F1B prescription for the reasons discussed in Sec. 2.3. The decisive metric is per-GPU compute efficiency: 696.9 TFLOP/s at 87B on 32 GPUs versus ∼681 TFLOP/s for the 16B single-node optimum. The reduction in tokens/sec/GPU at 87B reflects parameter count at fixed GBS rather than utilisation loss.

4.4. Summary across phases

We consolidate the empirical progression across all three optimization phases in Tab. 5.

Table 4. Phase 3 87B sweep, 8 nodes (32×GH200), TP=4 PP=4, 15-step throughput. VPP-off is optimal.

Config (87B, TP=4 PP=4)	tok/s/GPU	TFLOP/s/GPU
VPP-off (optimum)	1,299	696.9
MBS=2 + sel. recompute	1,278	685.8
VPP=7	1,146	615.0
VPP=2	877	470.2

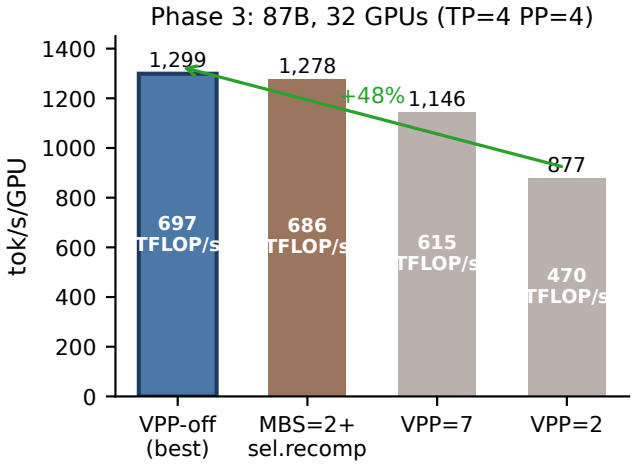

Figure 4. Phase 3 VPP sweep at 87B/TP=4/PP=4 on 8 nodes (cf. Table 4).

Table 5. Optimized configurations selected in each phase.

Config	Model	GPUs	tok/s/GPU	Total tok/s
Repo BF16 base.	8B	1	10,801	10,801
P1 optimum (FP8++)	8B	1	16,809	16,809
P2 optimum (TP=4)	16B	4	7,093	28,372
P3 16B (PP=2)	16B	8	5,158	41,264
P3 optimum (PP=4, VPP-off)	87B	32	1,299	41,568

4.5. Training convergence

We verify that the optimized configurations do not degrade training dynamics: the 87B 8-node run (100 steps) shows smooth, monotonic loss descent from ∼13.3 to 7.77 (Fig. 5), confirming that FP8 delayed scaling, TE cross-entropy, and VPP-off are loss-preserving. Interactive curves are available in the W&B dashboard (Run ID: `ls1c8yp5`).

4.6. Takeaways

Profile, don't guess. Every Phase 2/3 finding that contradicted the proposal or the framework's defaults — the 87B ceiling, the `tp-comm-overlap` cap, the width-vs-depth result, the negative result on selective recompute, and the VPP reversal at 87B — came from reading a profile or memory peak rather than trusting documented behaviour.

Wide models scale better across nodes. The 87B optimum preserves per-GPU compute efficiency (∼102% of the

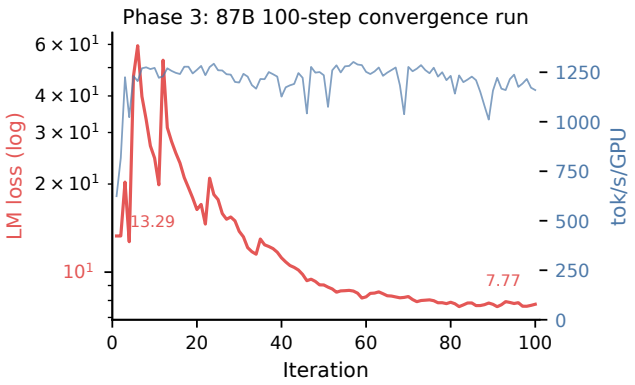

Figure 5. Training loss curve, 87B optimum, 100 steps.

16B value) across 32 GPUs, while the narrow 16B model lost 27% to PP P2P at a single node-boundary. Arithmetic intensity, not parameter count, determines how well a model hides inter-node latency.

The hardware imposes ceilings that no flag can lift. FP8 is load-bearing in Phase 1 both for GEMM speed and (transitively) the memory it frees. TP=4 is a memory tool, not a speed tool: 8B on 4 GPUs gets twice the per-node throughput at TP=1+DP=4 than at TP=4. And CUDA Multicast / the optimizer-state path each define real hardware-software ceilings on this stack, not tuning opportunities.

5. Artifacts

- Full NSYS traces and logs: https://drive.google.com/drive/folders/1Hd8J31Q_jho-Raace13yg7fODmEVaNLT?usp=sharing
- Wandb report: https://wandb.ai/mfornaini-eth-z-rich/gipfelsturm/reports/LSAIE-SS26--VmlldzoxNzA1MTgxMA
- All the other relevant logs and artifacts are included in the zip.